

UCS503P – Software Engineering Project

POTLI

On-Demand Luggage Storage Through Trusted Local Businesses

Submitted by:

1024030088 Satyam Tiwari
1024030525 Ishant Mehndiratta
1024030494 Anshaj
1024030498 Aayush Bindal

BE Third Year, Computer Science and Engineering

Submitted to:

Dr. Nisha Thakur

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Abstract

POTLI is a proposed web application for short-term luggage storage at local shops, hotels, and other businesses. It is intended for travelers who need to leave their bags between transport connections, accommodation check-in times, or other activities. The proposed workflow covers finding storage, reserving a time slot, and verifying drop-off and pickup through a QR code or one-time password (OTP). Storage partners would manage capacity and receive payment for completed bookings, while administrators would review partner applications and service records.

This report presents the requirements, system design, prototype implementation, and proposed evaluation. The prototype uses TypeScript and Vite for the frontend, Express for the API, Appwrite for authentication, and PostgreSQL with Drizzle ORM for application profiles. Account access and profile persistence are implemented; listing search, reservations, luggage handoffs, payments, ratings, and administrative operations remain to be developed. The proposed primary evaluation metric is Booking Fulfillment Time (BFT), with a pilot target of a median time no greater than three minutes.

Contents

Abstract	i
1 Introduction	1
1.1 Background and Context	1
1.2 Problem Statement	1
1.3 Stakeholders	1
1.4 Project Objectives	2
1.5 Scope and Current Status	2
2 Requirements and System Analysis	5
2.1 Functional Requirements	5
2.1.1 Traveler Requirements	5
2.1.2 Partner Requirements	5
2.1.3 Administrator Requirements	5
2.2 Non-Functional Requirements	5
2.3 Use-Case Model	6
2.4 Activity Model	7
3 Methodology and Design	9
3.1 Development Approach	9
3.2 High-Level Architecture	11
3.3 Data Flow Diagrams	11
3.3.1 Level 0: Context	11
3.3.2 Level 1: System Decomposition	12
3.3.3 Level 2: Booking and Handoff	14
3.4 Domain Model	16
3.5 Component Model	17
3.6 Behavioral Models	19
3.6.1 Sequence Diagram	19
3.6.2 Collaboration Diagram	21
3.6.3 State Diagram	23
4 Implementation Details	25
4.1 Frontend Prototype	25
4.2 Backend Authentication API	25
4.3 Database Schema	26
4.4 Security and Privacy Considerations	26

5	Testing and Evaluation	27
5.1	Testing Methodology	27
5.2	Current Verification Status	27
5.3	Evaluation Metrics	28
5.4	Pilot Plan	29
6	Conclusion	31
6.1	Summary of Work	31
6.2	Key Findings	31
6.3	Future Work and Recommendations	31
6.4	Final Remarks	32
	Bibliography	33

List of Figures

2.1	POTLI use-case diagram	6
2.2	End-to-end POTLI activity diagram	7
3.1	Project development timeline	10
3.2	Level 0 context data flow diagram	12
3.3	Level 1 system data flow diagram	13
3.4	Level 2 booking and luggage-handoff data flow diagram	15
3.5	Planned POTLI domain class diagram	16
3.6	POTLI implementation component diagram	18
3.7	Booking and handoff sequence diagram	20
3.8	Create-booking collaboration diagram	22
3.9	Booking lifecycle state diagram	23

List of Tables

1.1	Implemented and planned project scope	3
5.1	Mid-semester verification status	28
5.2	Proposed evaluation metrics	28

Introduction

1.1 Background and Context

Travelers may need temporary luggage storage before hotel check-in, after check-out, or while waiting for a transport connection. Carrying bags during these intervals limits movement and access to other activities. Storage facilities at stations and airports may not be near the traveler's destination, and availability may be difficult to establish in advance. Local businesses offer a possible alternative where suitable storage space is available.

POTLI is designed to connect travelers with participating businesses and maintain a record of each storage booking. Partners would publish their capacity, opening hours, and prices. The system would record both luggage handoffs, and administrators would review partner details and disputed bookings. Finding a location is therefore only one part of the proposed service; reservation and handoff records are also required.

1.2 Problem Statement

The project addresses the difficulty of finding suitable short-duration luggage storage near a traveler's destination. An informal arrangement may not provide a booking receipt, an availability check, or a consistent record of custody. The proposed system must make these records available without imposing a lengthy procedure on either the traveler or the storage partner.

The interface must be usable on ordinary smartphones and understandable to business owners without technical training. Intermittent connectivity is a further design consideration, particularly during physical handoffs. The project uses standard web technologies and does not require dedicated storage hardware.

1.3 Stakeholders

- **Traveler:** searches, compares locations, books a slot, presents handoff credentials, tracks status, and rates the experience.
- **Storage partner:** registers a business, publishes capacity and hours, confirms drop-off and pickup, and views earnings.

- **Administrator:** verifies partners, audits bookings and fees, reviews reports, and resolves flagged cases.
- **Payment gateway:** processes collections, refunds, payouts, and transaction status in the planned system.

1.4 Project Objectives

1. Deliver a responsive web interface for traveler discovery, account access, partner onboarding, and role-specific operations.
2. Implement secure role-based authentication and a persistent data model for travelers, storage partners, and administrators.
3. Build a booking lifecycle with availability validation, QR/OTP-based drop-off and pickup, timestamps, and photographic handoff evidence.
4. Instrument the workflow so that median BFT and handoff success can be measured objectively.
5. Prepare the system for payment settlement, administrative oversight, repeatable deployment, and later geographic scaling.

1.5 Scope and Current Status

Table 1.1 summarizes the prototype’s current scope. The design diagrams also cover features scheduled for later development.

Capability	Status	Evidence
Landing and account-access pages	Implemented	Landing page, role-specific login, partner registration form, and account summary pages
Versioned authentication API	Implemented	/api/v1/health, /api/v1/auth/login, and /api/v1/auth/me
Account and profile persistence	Implemented	Appwrite accounts; PostgreSQL <code>profiles</code> and <code>partner_profiles</code>
Search and listing availability	Planned	Modeled by DFD process 2.0 and UML interactions
Booking and QR/OTP handoff	Planned	Modeled by DFD processes 3.1–3.5
Payments, payouts, ratings, and disputes	Planned	Included in approved design and later deliverables

Table 1.1: Implemented and planned project scope

Requirements and System Analysis

2.1 Functional Requirements

2.1.1 Traveler Requirements

A traveler should be able to search by city, landmark, or proximity; compare price, opening hours, and availability; reserve a time window; receive a booking identifier and QR/OTP; verify both physical handoffs; review current and prior bookings; and submit a post-pickup rating.

2.1.2 Partner Requirements

A partner should be able to create a business profile, submit details for verification, define capacity and operating hours, receive reservations, validate handoff credentials, attach a drop-off photograph, restore capacity after pickup, and review earnings.

2.1.3 Administrator Requirements

An administrator should be able to approve or reject partner applications, inspect users and listings, monitor booking states, audit payment and platform-fee records, review performance summaries, and investigate disputes or suspicious listings.

2.2 Non-Functional Requirements

- **Usability:** short forms, clear role-specific screens, and a mobile-responsive interface.
- **Security:** salted password hashes, signed expiring tokens, authorization checks, server-side validation, secret management, and protected handoff evidence. Authentication controls should follow established web guidance [OWA25].
- **Reliability:** explicit booking states and idempotent state transitions to avoid duplicate confirmations.
- **Performance:** indexed database queries, pagination, compressed assets, and a median BFT target of at most three minutes.
- **Scalability:** stateless API instances and a database structure that can later support indexed location queries.

- **Maintainability:** versioned endpoints, migration scripts, source-controlled diagram definitions, and clear separation of frontend, API, and data layers.

2.3 Use-Case Model

Figure 2.1 places traveler, partner, and administrator interactions inside one system boundary. It demonstrates that booking depends on discovery and that the trust process continues through both handoffs and administrative oversight.

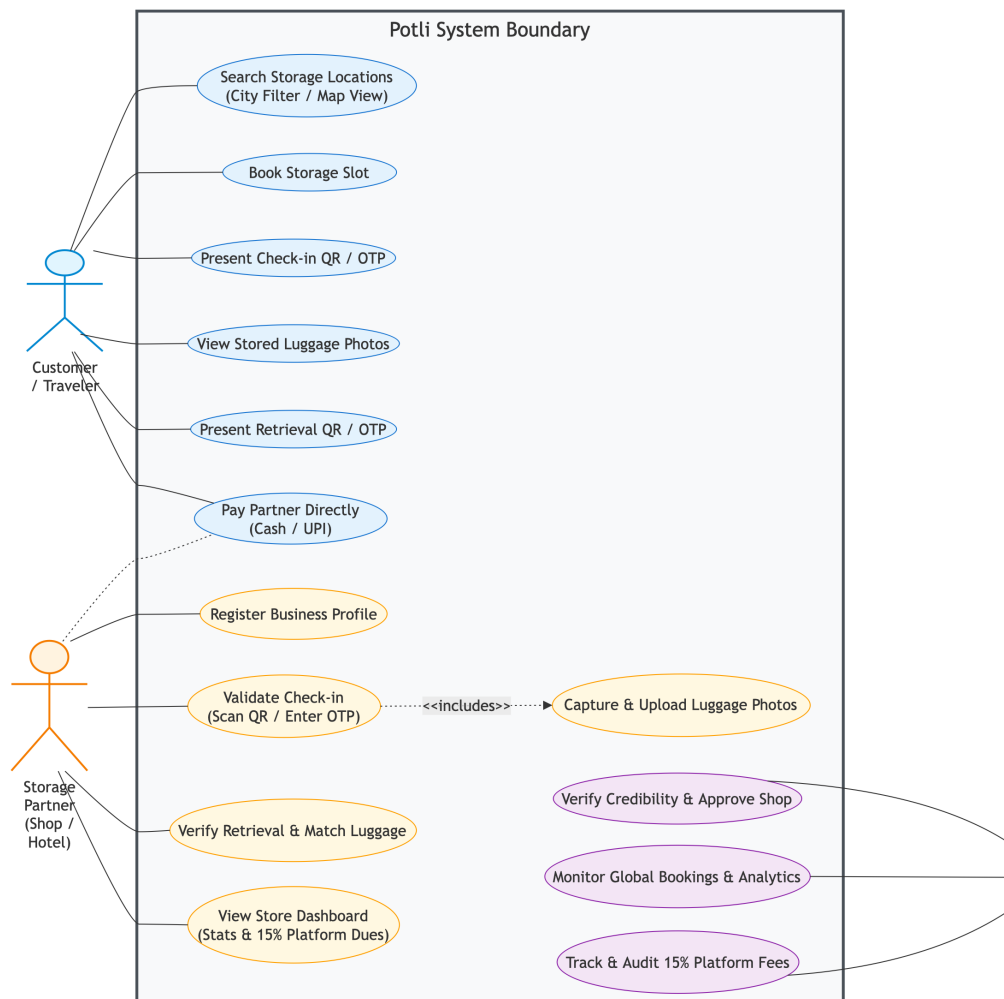


Figure 2.1: POTLI use-case diagram

Methodology and Design

3.1 Development Approach

Development is organized into increments. The first establishes the frontend, authentication API, and profile database. The next introduces storage listings and search, followed by reservations and luggage handoffs. Partner and administrator operations, payment processing, testing, and deployment follow these core functions.

Figure 3.1 presents the approved development schedule from requirements analysis to the launch demonstration. It records planned activities, not their completion status.

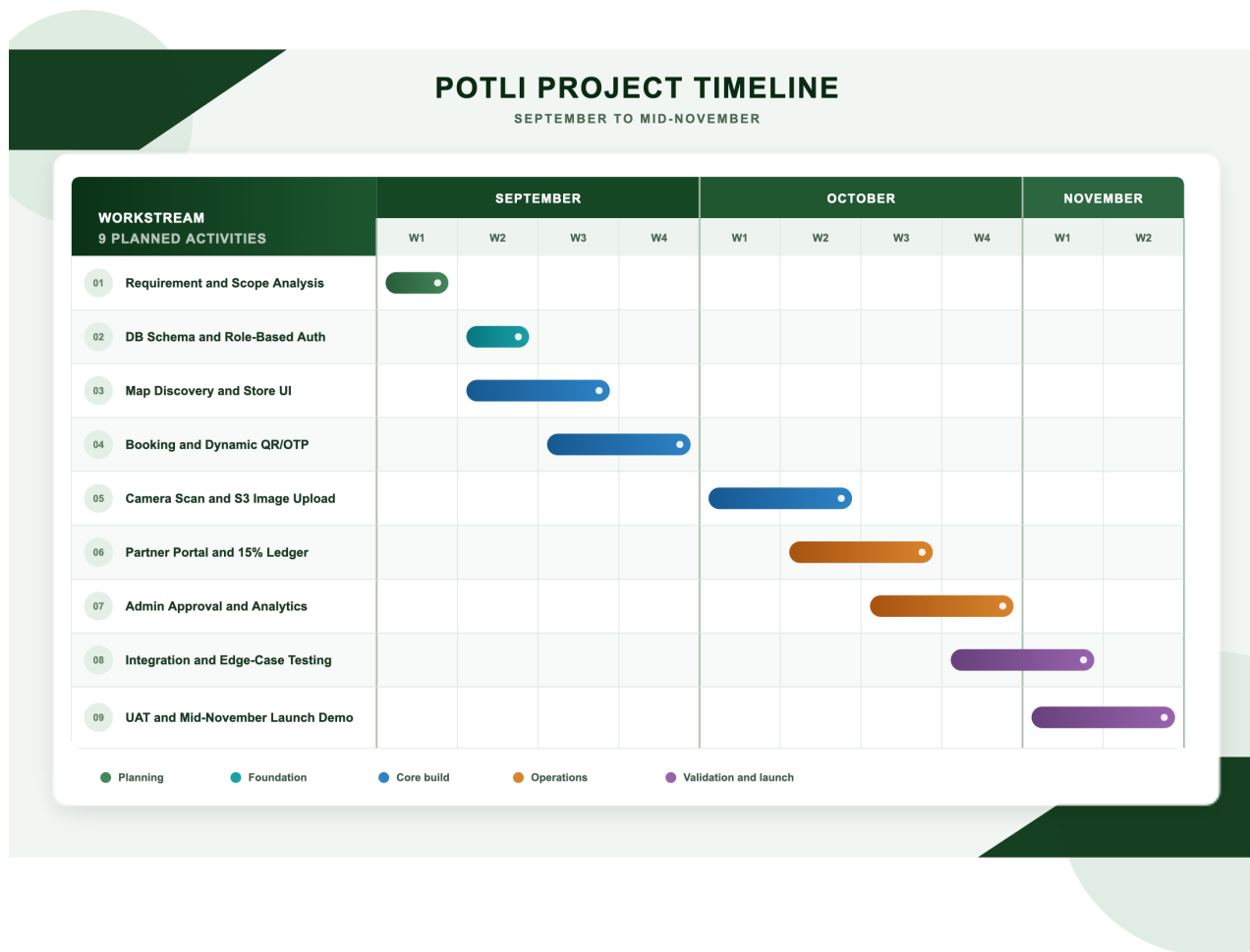


Figure 3.1: Project development timeline

3.2 High-Level Architecture

POTLI uses a three-tier web architecture:

1. **Presentation layer:** a TypeScript application bundled with Vite [Vit25]. It renders the public and account pages and uses Appwrite's Web SDK for authentication.
2. **Application layer:** an Express [Ope25] API running on Node.js. The current endpoints are grouped under `/api/v1` and provide health checks, login, and authenticated profile access.
3. **Data layer:** PostgreSQL [Pos25], accessed through a shared Drizzle ORM [Dri] workspace and the `pg` driver. Application roles and partner details are stored here. Appwrite [App] manages identity and credentials separately.

The browser signs in to Appwrite and requests a JSON Web Token (JWT) [JBS15] for the current session. It sends this token to the backend using the Bearer scheme. The backend asks Appwrite to identify the account and retrieves the corresponding application profile from PostgreSQL. An account without a profile receives a default traveler profile on its first authenticated profile request. Non-browser clients can use the backend login endpoint. Search, booking, photo storage, and payment services have not yet been added.

3.3 Data Flow Diagrams

3.3.1 Level 0: Context

The Level 0 DFD (Figure 3.2) treats POTLI as a single process. Registration, search, booking, business updates, administration, and payment data cross the system boundary through four external entities.

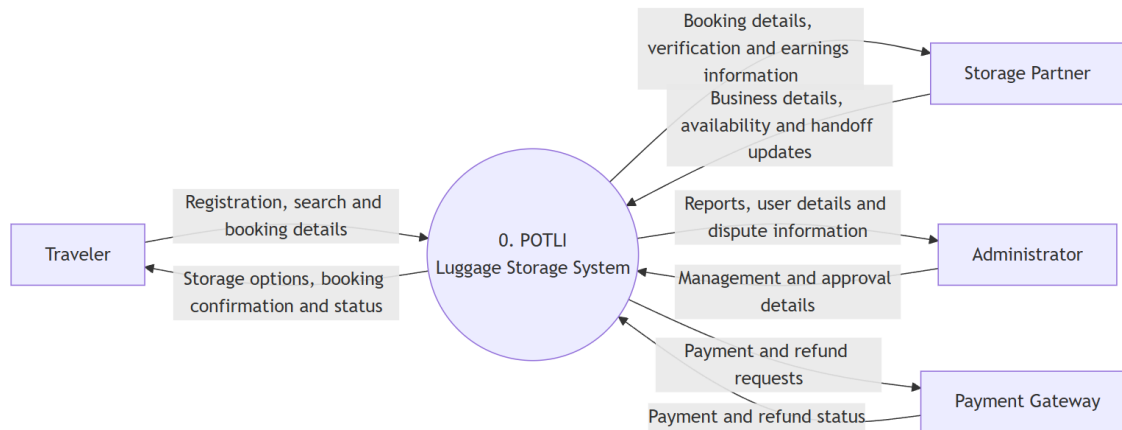


Figure 3.2: Level 0 context data flow diagram

3.3.2 Level 1: System Decomposition

Figure 3.3 decomposes the platform into account management (1.0), storage listing management (2.0), bookings and handoffs (3.0), payment processing (4.0), and platform administration (5.0). Four conceptual stores hold user/partner data, listings, bookings, and payments.

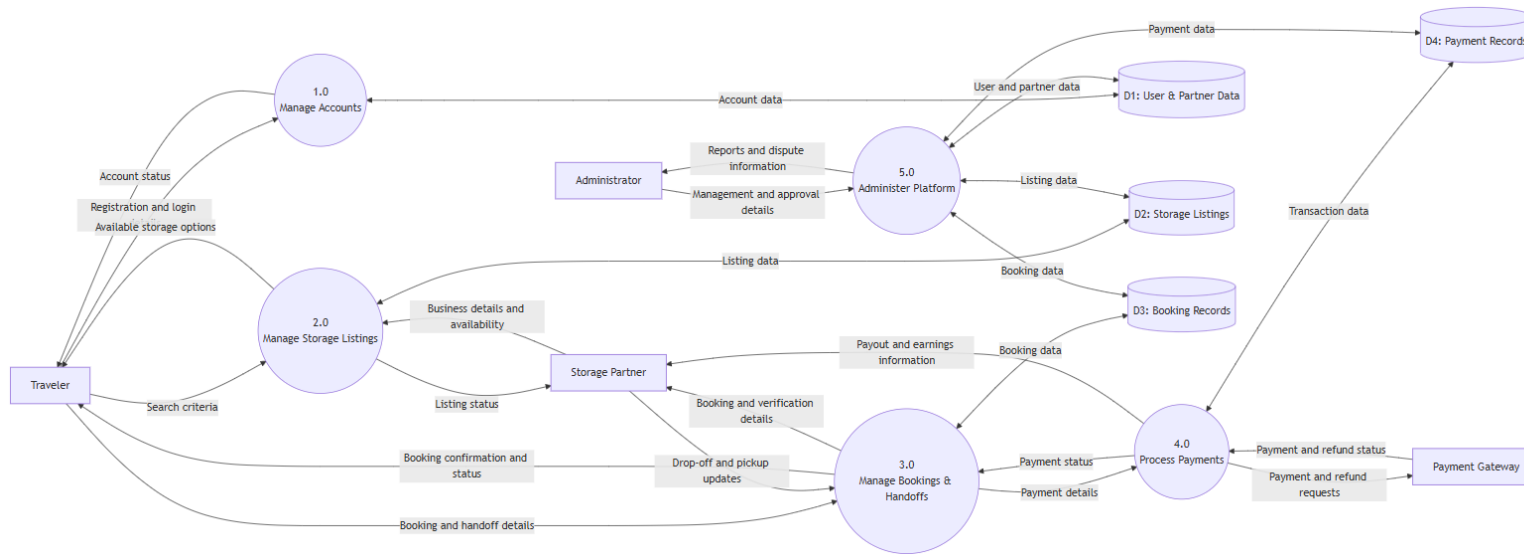


Figure 3.3: Level 1 system data flow diagram

3.3.3 Level 2: Booking and Handoff

The detailed DFD in Figure 3.4 expands process 3.0. A request is validated against listing data, a booking is created after payment status is received, the drop-off is verified and recorded, and pickup completes the booking while restoring capacity.

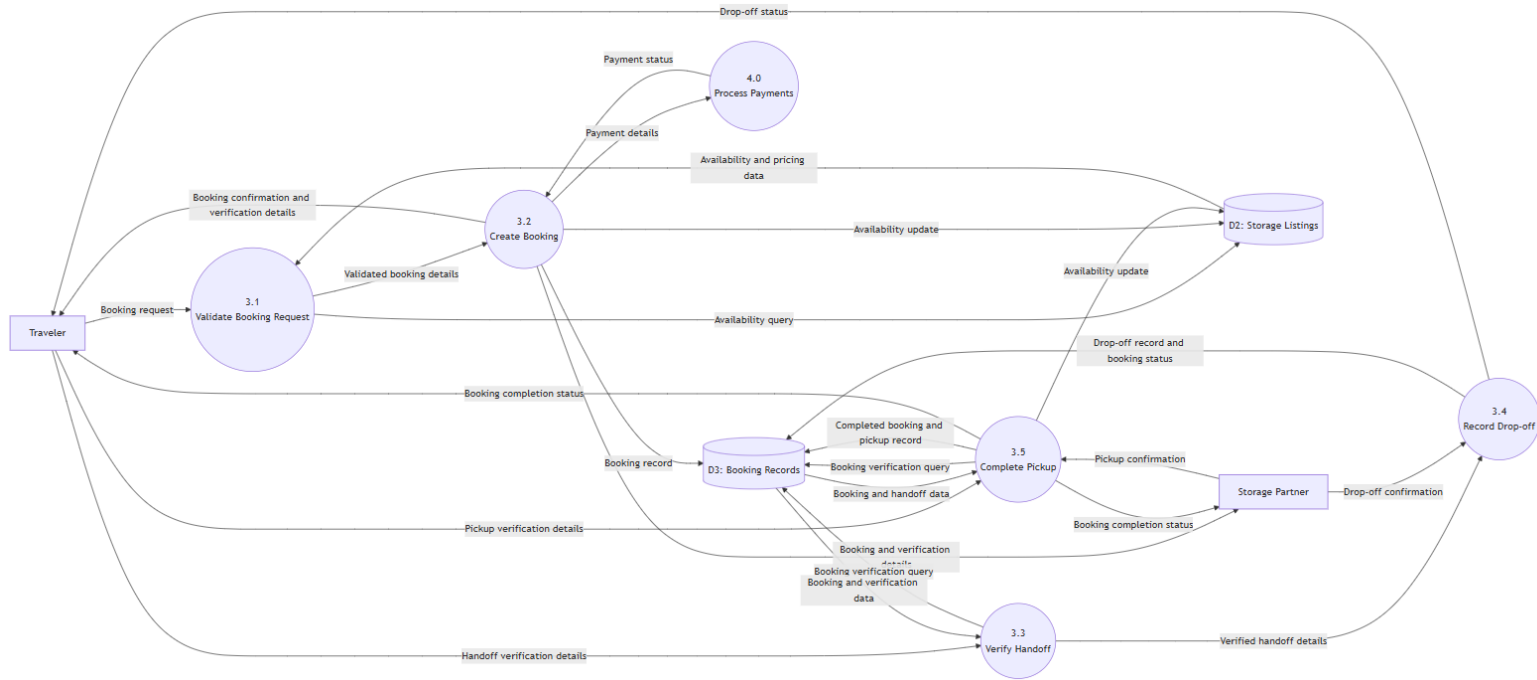


Figure 3.4: Level 2 booking and luggage-handoff data flow diagram

3.4 Domain Model

The class model in Figure 3.5 connects the common **User** identity to traveler, partner, and administrator roles. It separates a partner's business profile from storage listings and composes booking records with handoff and payment information. Ratings remain optional because they occur after successful pickup.



Figure 3.5: Planned POTLI domain class diagram

3.5 Component Model

The component diagram in Figure 3.6 describes the current implementation rather than the planned business workflow. The browser application uses the backend REST API and Appwrite’s Web SDK. The backend depends on configuration, an identity adapter, and the shared database workspace. Appwrite provides account services, while PostgreSQL stores application profiles. Dashed arrows indicate dependencies; circles and semicircles identify provided and required interfaces.

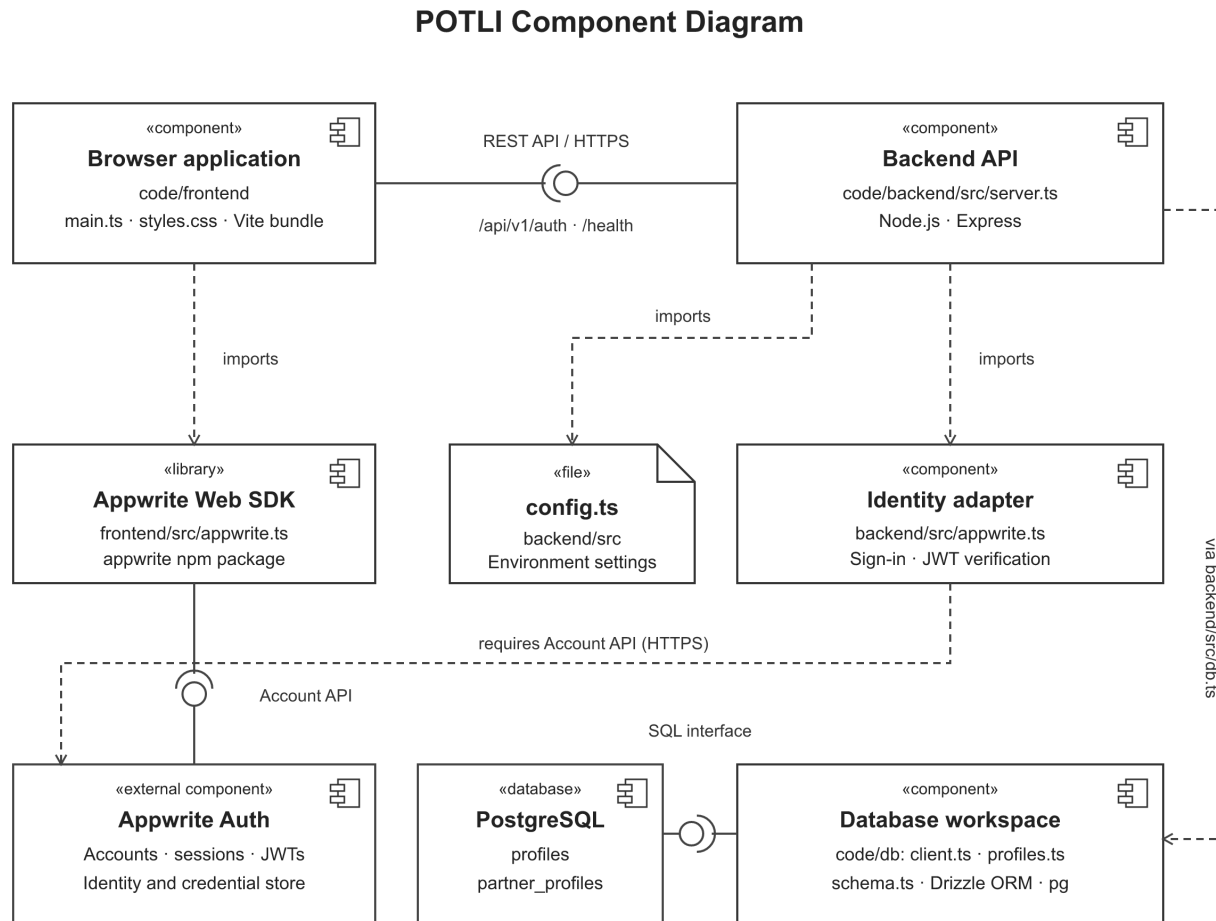


Figure 3.6: POTLI implementation component diagram

3.6 Behavioral Models

3.6.1 Sequence Diagram

Figure 3.7 shows the proposed interaction among the traveler, web application, back-end, database, payment gateway, and partner. It covers discovery, authentication, booking, handoff, settlement, and rating. The authentication exchange is a design-level view; the current implementation delegates identity checks to Appwrite.

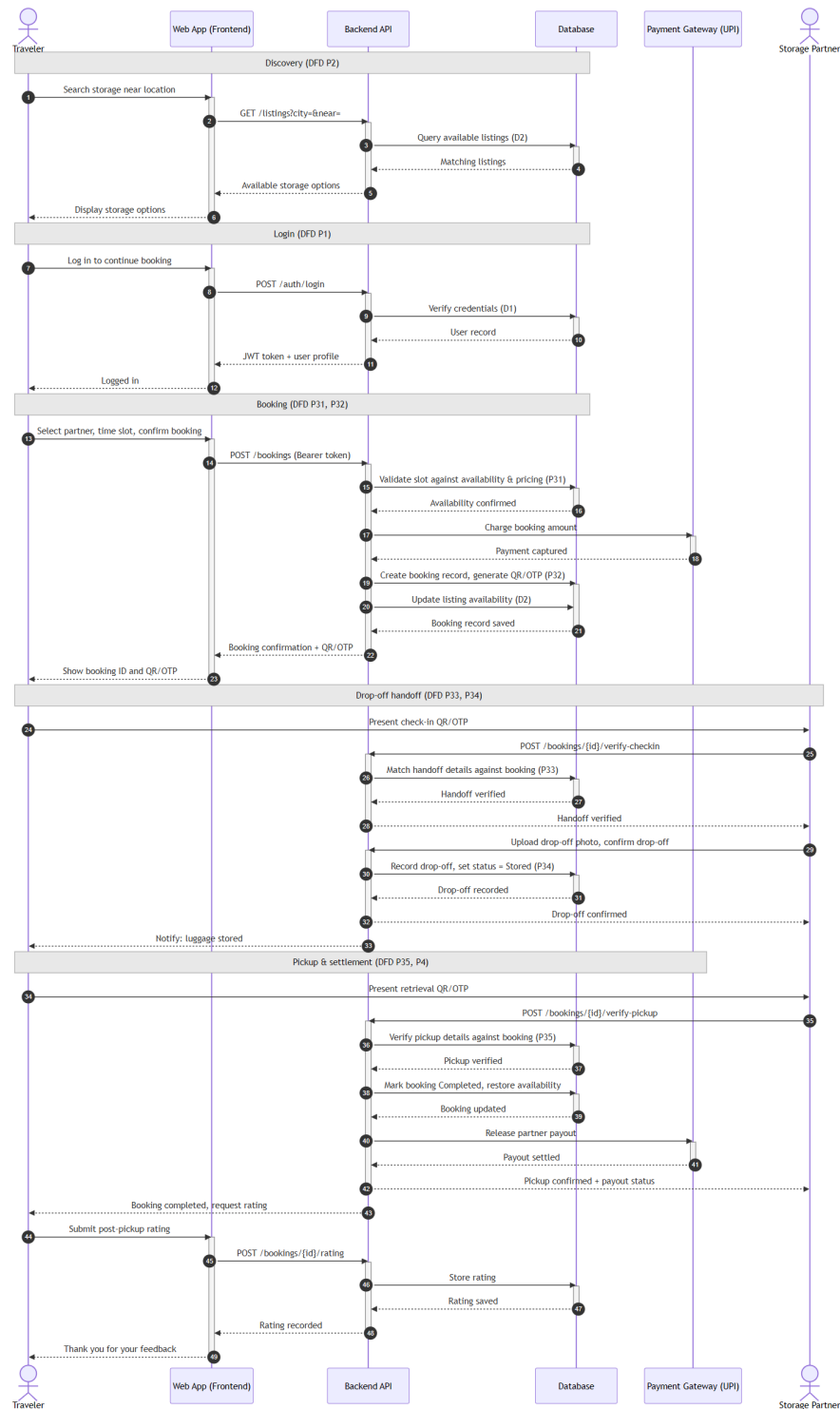


Figure 3.7: Booking and handoff sequence diagram

3.6.2 Collaboration Diagram

The collaboration diagram (Figure 3.8) presents the booking phase as numbered object-to-object messages rather than a time axis. It emphasizes the links among the web app, booking service, listing store, payment gateway, booking store, and partner.

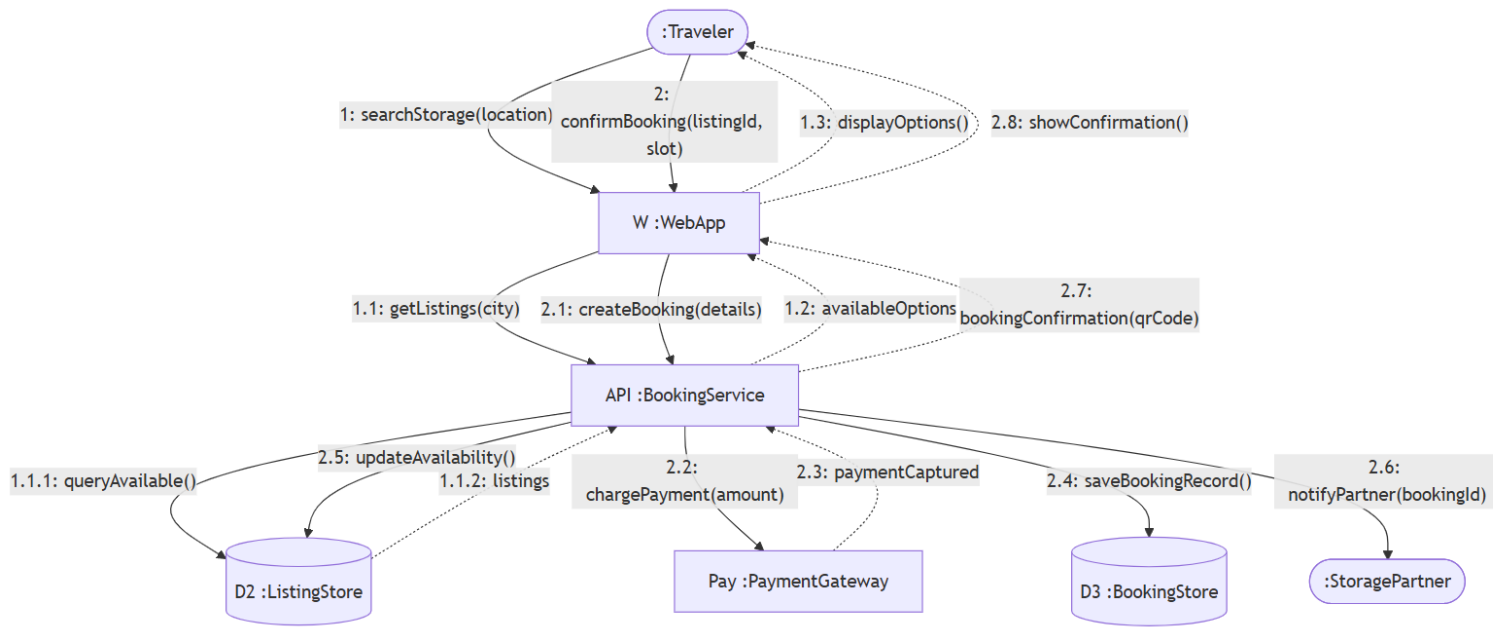


Figure 3.8: Create-booking collaboration diagram

3.6.3 State Diagram

The booking state machine in Figure 3.9 makes valid transitions explicit. A validated request enters **PendingPayment**; successful capture creates **Booked**; verified handoffs lead through **Dropped**, **Stored**, and **PickedUp**; payout settlement produces **Completed**. Failed payment, cancellation, or a missed slot enters a terminal exception state.

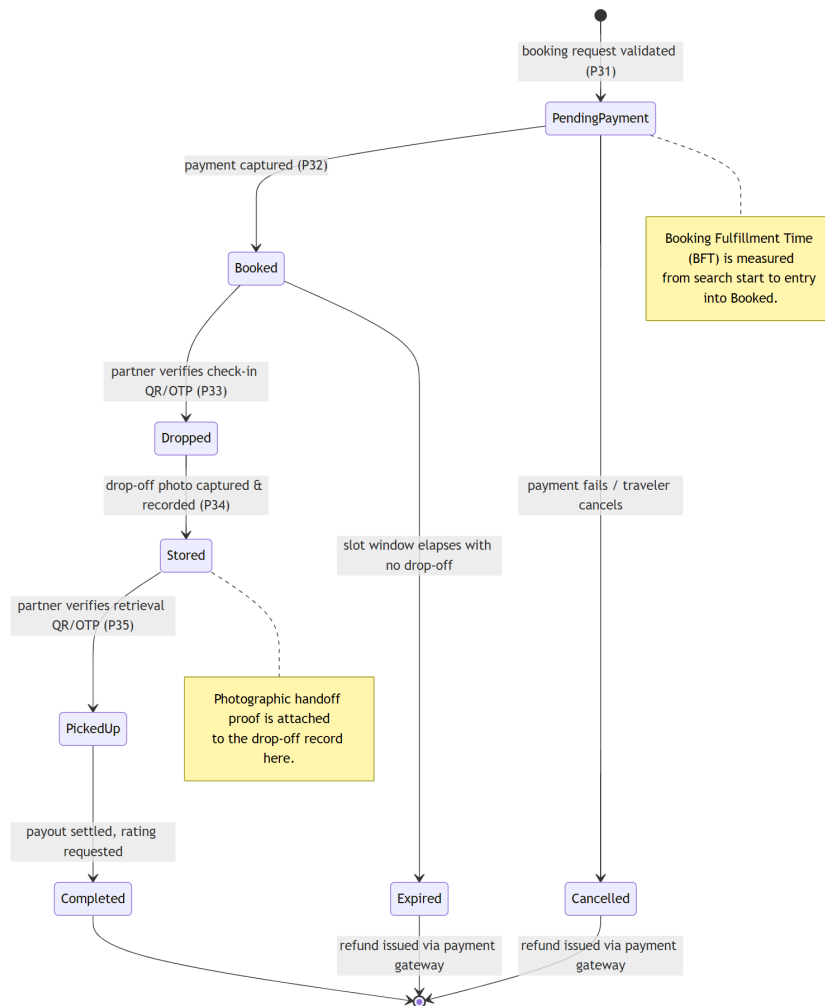


Figure 3.9: Booking lifecycle state diagram

Implementation Details

4.1 Frontend Prototype

The frontend is written in TypeScript, bundled with Vite, and styled with a responsive stylesheet. A single rendering entry point selects the page from the URL hash. The available pages cover the landing page, traveler and administrator login, partner account access, password recovery, and account summaries. Shared SVG icons and consistent form labels are used across these pages.

Login forms use Appwrite's Web SDK to establish a session. The frontend then requests a JWT and calls `/api/v1/auth/me` to obtain the user's profile and role. Each login page checks that the returned role matches the selected account type. This controls page navigation; it does not replace server-side authorization for future booking or administrative operations. Search submission and partner registration are not yet connected to persistent records.

Password-recovery and reset forms call Appwrite's recovery endpoints. The reset page reads the account identifier and recovery secret from the emailed link and removes them from the URL after a successful reset. Email delivery requires the Appwrite mail service to be configured.

4.2 Backend Authentication API

The backend uses TypeScript on Node.js and Express with CORS and JSON middleware. Its main implemented responsibilities are:

- checking PostgreSQL connectivity and Appwrite availability through `GET/api/v1/health`;
- submitting email/password credentials to Appwrite and returning a JWT through `POST/api/v1/auth/login`; and
- identifying the caller through Appwrite, loading or creating the application profile, and returning it through `GET/api/v1/auth/me`.

The backend identity adapter calls Appwrite's Account API. For server-side login, it carries the session cookie returned by Appwrite to the JWT endpoint. For authenticated requests, it sends the caller's JWT to Appwrite to obtain the account identity. It does not store passwords or implement token signing locally.

The backend database module provides access to the shared Drizzle workspace. A central error handler receives failures from asynchronous routes. All current API routes are versioned.

4.3 Database Schema

The Drizzle schema defines two application tables. The **profiles** table has a generated UUID primary key, a unique **appwrite_user_id**, email, full name, role, and creation and update timestamps. The role is restricted to **traveler**, **storage_partner**, or **admin**. The **partner_profiles** table stores business name and phone number and references the local profile UUID through a cascading foreign key.

Appwrite accounts are linked by their string identifiers, not by a foreign key to an authentication table. A profile is created by the backend when an authenticated account first requests profile access. This path assigns the traveler role; users cannot supply a different role through that endpoint. The current migration does not define signup triggers or row-level security policies.

SQL migrations are applied in filename order, with completed versions recorded in a **migrations** table. The development seed script creates or updates Appwrite accounts through the Users API and writes their roles and partner details to PostgreSQL. Seeding requires a server-side Appwrite API key as well as database access.

Later migrations will add the D2–D4 stores for listings, capacity, bookings, handoffs, and transactions. Availability checks and reservation writes must be performed transactionally to prevent concurrent bookings from consuming the same capacity.

4.4 Security and Privacy Considerations

Appwrite manages passwords, sessions, and JWT issuance. The backend identifies a caller before returning the caller’s application profile, and it assigns a default role when creating that profile. Database credentials and the administrative API key used for seeding must remain server-side.

Further work is required before public deployment. Protected domain endpoints need server-side role checks, request validation, rate limiting, and audit records. CORS settings must be restricted to the deployed frontend, and error responses must avoid exposing internal database details. Session expiry, revocation, and recovery-link handling also require integration tests. Handoff photographs and personal information will need access controls and defined retention periods. These are requirements for subsequent development, not controls established by the present prototype.

Testing and Evaluation

5.1 Testing Methodology

The planned verification strategy combines:

- **Unit tests** for validation, status transitions, pricing, token handling, and utility functions;
- **API integration tests** against an isolated PostgreSQL database for login, authorization, booking races, handoff codes, and invalid transitions;
- **Frontend tests** for route rendering, form behavior, error handling, responsive layout, and accessibility;
- **End-to-end tests** for traveler booking through partner pickup and administrator review;
- **Performance tests** for nearby search and concurrent booking creation; and
- **Pilot user testing** with representative travelers and local businesses.

5.2 Current Verification Status

The repository provides scripts for TypeScript checking, frontend production builds, SQL migrations, and development-account seeding. The checks listed in Table 5.1 are available development tools; the table does not report test results. Live authentication checks require both Appwrite and PostgreSQL, while recovery-email tests also require a configured mail service. An automated backend test suite and the complete booking workflow are not yet available. No pilot performance measurements have been collected.

Check	Status	Interpretation
TypeScript compilation and Vite production build	Available	Detects type/build failures and produces deployable static assets
Database migration and seed scripts	Available	Applies SQL migrations; seeding also requires Appwrite access and an administrative API key
Automated backend unit/integration suite	Not yet implemented	Needed to verify protected operations and error cases
End-to-end booking workflow	Not yet testable	Booking and handoff modules remain planned
Pilot performance measurements	Not yet collected	Must follow implementation and field trial

Table 5.1: Mid-semester verification status

5.3 Evaluation Metrics

Metric	Target	Measurement
Median Booking Fulfillment Time	≤ 3 minutes	Search-start timestamp to confirmed-booking timestamp
Handoff success rate	Establish after pilot	Completed drop-off and pickup without dispute / initiated handoffs
Partner utilization	Establish baseline	Booked capacity / listed capacity per partner-week
Payout turnaround	Define with gateway	Completion timestamp to credited-payout timestamp
Traveler satisfaction	1–5 scale	Post-pickup rating and short qualitative feedback
Business-hour availability	$\geq 99\%$ pilot goal	Successful health/search probes during defined hours

Table 5.2: Proposed evaluation metrics

5.4 Pilot Plan

A small locality near a railway station or busy market can provide a controlled first deployment. The proposed pilot includes 8–12 partner businesses and 15–20 traveler participants. Before use, partners receive a brief handoff procedure and the team confirms business details. Application timestamps provide quantitative measures, while structured post-use questions capture convenience, trust, and onboarding difficulty. Any safety incident or disputed handoff takes priority over performance measurement.

Conclusion

6.1 Summary of Work

The work completed so far includes requirements analysis, the system architecture, data flow and UML models, a responsive frontend, a versioned authentication API, and an application profile schema. The prototype connects Appwrite account access with PostgreSQL profile persistence.

The prototype does not yet provide a complete luggage-storage service. Listing search, reservations, QR/OTP handoffs, payments, ratings, and administrative operations remain to be implemented and tested.

6.2 Key Findings

- A booking record must retain the information needed to verify custody: handoff credentials, timestamps, and photographic evidence.
- Availability and booking creation form the central consistency boundary; they must be updated transactionally to prevent overbooking.
- Account identity is managed by Appwrite. PostgreSQL stores application roles and business records.
- The BFT target is useful only when timestamps and state definitions are implemented before pilot measurement.

6.3 Future Work and Recommendations

1. Implement listing management and indexed location search.
2. Add transactional booking creation and dynamic QR/OTP credentials.
3. Build protected partner handoff screens with secure photo upload.
4. Introduce role-authorization middleware and comprehensive automated tests.
5. Integrate a sandbox payment gateway before enabling real collections or payouts.
6. Build administrator verification, audit, analytics, and dispute tools.

7. Deploy a staging environment, run the proposed pilot, and replace target values with measured results.

6.4 Final Remarks

The next milestone is a tested workflow from listing search to verified pickup. It must include a reliable capacity check, a persistent booking record, and both handoff confirmations. Completing this workflow will allow the team to measure BFT and evaluate the service with the proposed pilot participants.

Bibliography

- [App] Appwrite. *Appwrite Documentation*. URL: <https://appwrite.io/docs> (cit. on p. 11).
- [Dri] Drizzle Team. *Drizzle ORM Documentation*. URL: <https://orm.drizzle.team/docs/overview> (cit. on p. 11).
- [JBS15] Michael Jones, John Bradley, and Nat Sakimura. *JSON Web Token (JWT)*. Tech. rep. RFC 7519. Internet Engineering Task Force, 2015 (cit. on p. 11).
- [Ope25] OpenJS Foundation. *Express: Fast, unopinionated, minimalist web framework for Node.js*. 2025. URL: <https://expressjs.com/> (visited on 10/06/2025) (cit. on p. 11).
- [OWA25] OWASP Foundation. *Authentication Cheat Sheet*. 2025. URL: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html (visited on 10/06/2025) (cit. on p. 5).
- [Pos25] PostgreSQL Global Development Group. *PostgreSQL Documentation*. 2025. URL: <https://www.postgresql.org/docs/> (visited on 10/06/2025) (cit. on p. 11).
- [Vit25] Vite Contributors. *Vite: Next Generation Frontend Tooling*. 2025. URL: <https://vite.dev/> (visited on 10/06/2025) (cit. on p. 11).